

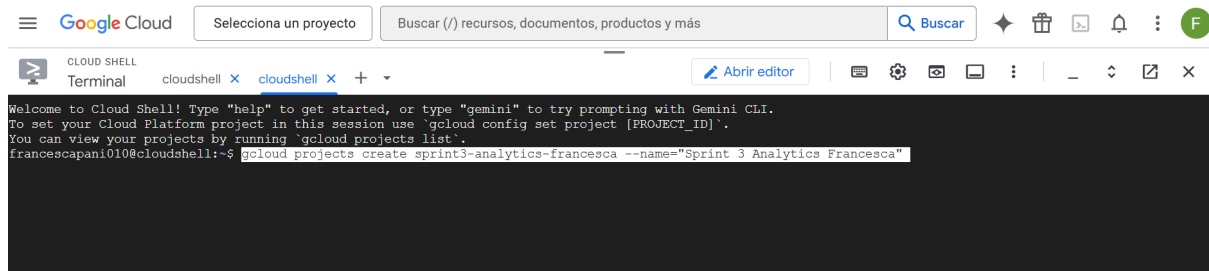
Nivel 1: Entorno e Ingesta Híbrida (Code-First)

Ejercicio 1: Arquitectura de Datos (Lógica vs. Física)

Crea un Nuevo Proyecto en Google Cloud llamado: *sprint3-analytics-[el-teu-nom]*.

Puedes crearlo por la UI, pero si lo haces por Consola (gcloud CLI) tiene un plus.

Empiezo creando el proyecto a través de la Consola (gcloud CLI) en lugar de la interfaz gráfica porque, aunque ambas opciones produzcan el mismo resultado, la línea de comandos aporta una ventaja importante: el comando es reproducible. Además, es el método recomendado por el enunciado. Ejecuto el siguiente comando en Cloud Shell:



```
Google Cloud
Selecciona un proyecto
Buscar (/) recursos, documentos, productos y más
Buscar

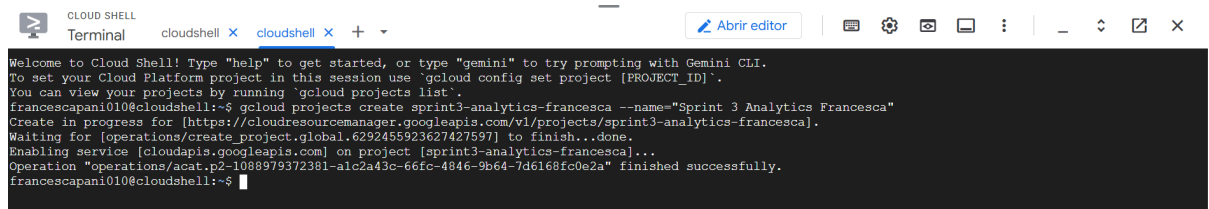
CLOUD SHELL
Terminal cloudshell x cloudshell x +
Abrir editor

Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use 'gcloud config set project [PROJECT_ID]'.
You can view your projects by running 'gcloud projects list'.
francescapani010@cloudshell:~$ gcloud projects create sprint3-analytics-francesca --name="Sprint 3 Analytics Francesca"
```

El comando se compone de:

- **gcloud projects create**: comando de la CLI de Google Cloud que crea un nuevo proyecto.
- **sprint3-analytics-francesca**: es el identificador único del proyecto (project ID).
- **--name="Sprint 3 Analytics Francesca"**: nombre legible que se muestra en la interfaz.

Esta es la salida, el proyecto se ha creado correctamente:



```
CLOUD SHELL
Terminal cloudshell x cloudshell x +
Abrir editor

Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use 'gcloud config set project [PROJECT_ID]'.
You can view your projects by running 'gcloud projects list'.
francescapani010@cloudshell:~$ gcloud projects create sprint3-analytics-francesca --name="Sprint 3 Analytics Francesca"
Create in progress for [https://cloudresourcemanager.googleapis.com/v1/projects/sprint3-analytics-francesca].
Waiting for [operations/create.project.global.6292455923627427597] to finish...done.
Enabling service [cloudapis.googleapis.com] on project [sprint3-analytics-francesca]...
Operation "operations/acet.p2-1088979372381-alc2a43c-66fc-4846-9b64-7d6168fc0e2a" finished successfully.
francescapani010@cloudshell:~$
```

Ahora verifico que el proyecto existe ejecutando **gcloud projects list**, ya que es una buena práctica verificar antes de seguir:



```
CLOUD SHELL
Terminal cloudshell x cloudshell x +
Abrir editor

Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use 'gcloud config set project [PROJECT_ID]'.
You can view your projects by running 'gcloud projects list'.
francescapani010@cloudshell:~$ gcloud projects create sprint3-analytics-francesca --name="Sprint 3 Analytics Francesca"
Create in progress for [https://cloudresourcemanager.googleapis.com/v1/projects/sprint3-analytics-francesca].
Waiting for [operations/create.project.global.6292455923627427597] to finish...done.
Enabling service [cloudapis.googleapis.com] on project [sprint3-analytics-francesca]...
Operation "operations/acet.p2-1088979372381-alc2a43c-66fc-4846-9b64-7d6168fc0e2a" finished successfully.
francescapani010@cloudshell:~$ gcloud projects list
PROJECT ID: sprint3-analytics-francesca
NAME: Sprint 3 Analytics Francesca
PROJECT NUMBER: 1088979372381
ENVIRONMENT:
francescapani010@cloudshell:~$
```

Ahora voy a configurar el proyecto (tal y como recomendado en el mensaje de bienvenida: "To set your Cloud Platform project in this session use gcloud config set project [PROJECT_ID]") para que, al lanzar los comandos sucesivos, no tenga que indicar el proyecto a mano cada vez:

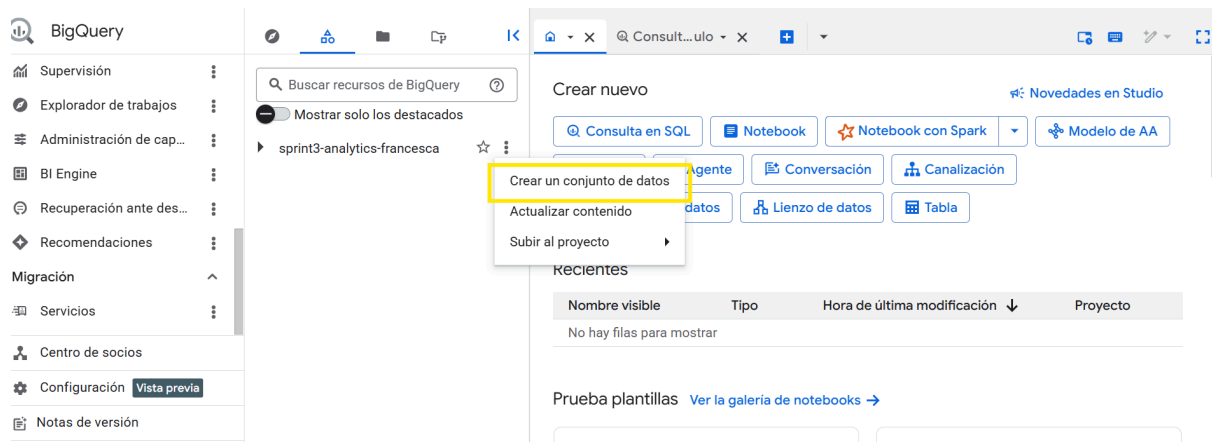
```
PROJECT_ID: sprint3-analytics-francesca
NAME: Sprint 3 Analytics Francesca
PROJECT_NUMBER: 1088979372381
ENVIRONMENT:
francescapani010@cloudshell:~$ gcloud config set project sprint3-analytics-francesca
Updated property [core/project].
francescapani010@cloudshell:~ (sprint3-analytics-francesca) $
```

Para demostrar tu dominio total de la plataforma, crearás cada uno de los datasets utilizando un método distinto:

- **Dataset Físico sprint3_bronze → Capa Lógica Bronce (Datos Crudas)**
 - Función: Datos crudos tal y como llegan de la fuente.
 - Método: Utiliza la Interfaz Gráfica (UI) de BigQuery (haciendo clics).
- **Dataset Físico sprint3_silver → Capa Lógica Silver (Datos Limpios)**
 - Función: Datos limpios, tipados y deduplicados.
 - Método: Utiliza código SQL (CREATE SCHEMA).
- **Dataset Físico sprint3_gold → Capa Lógica Gold (Datos de Negocio)**
 - Función: Datos agregados listos para informes y paneles de control (dashboards).
 - Método: Utiliza Cloud Shell (Línea de comandos bq).

Ten en cuenta la ubicación de los datos originales para decidir la ubicación física de los tres Datasets. Nota : Aunque sean capas lógicas diferentes, físicamente deben vivir en la misma región para poder "hablar" entre ellas sin costes de transferencia.

Cierro Cloud Shell y voy a seleccionar el proyecto que acabo de crear. Empiezo a crear el el dataset **sprint3_bronze** con la interfaz gráfica:



- ID del proyecto definido anteriormente: sprint3-analytics-francesca
- ID del conjunto de datos: sprint3_bronze
- Tipo de ubicación: Multi-región (como indicado en el enunciado)
- Multi-región: EU

Crear un conjunto de datos

ID del proyecto *
sprint3-analytics-francesca [Cambiar](#)

ID de conjunto de datos *
sprint3_bronze

Puede incluir letras, números y guiones bajos

Ubicación de los datos
EU (múltiples regiones en la Unión Europea) [?](#)

Conjunto de datos externo

The selected region supports the following external dataset types: Cloud Spanner

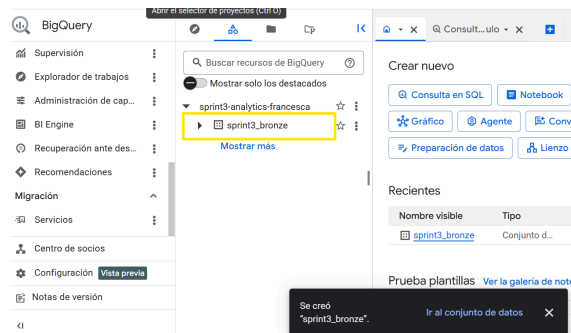
☐ Vínculo a un conjunto de datos externo [?](#)

Etiquetas [v](#)

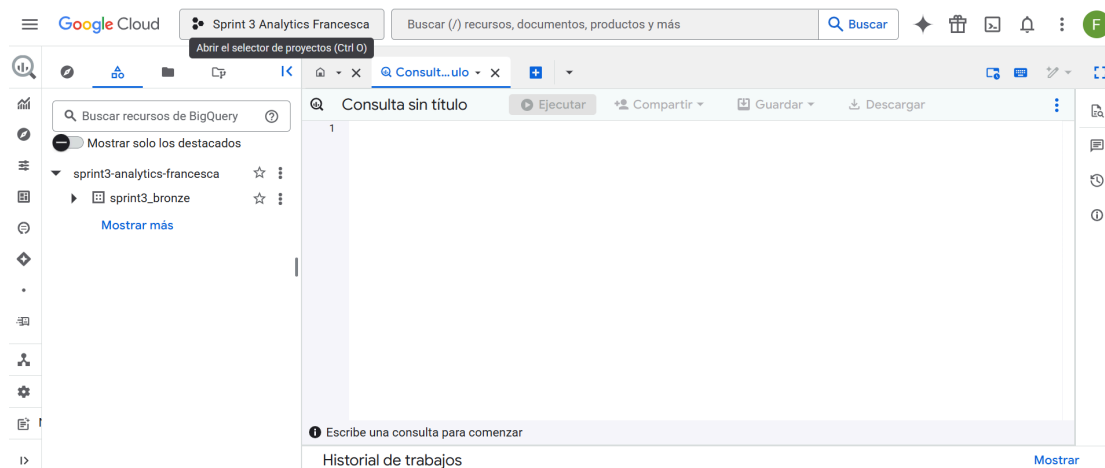
Opciones avanzadas [v](#)

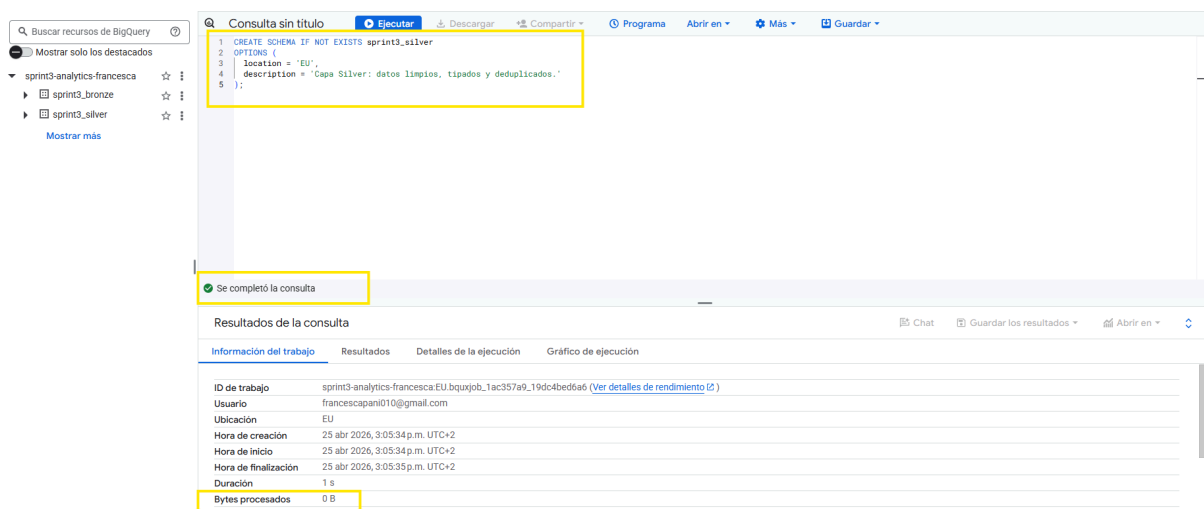
[Crear conjunto de datos](#) [Cancelar](#)

Dataset creado correctamente:



Sigo con **sprint3_silver**, que he de crear con código SQL.

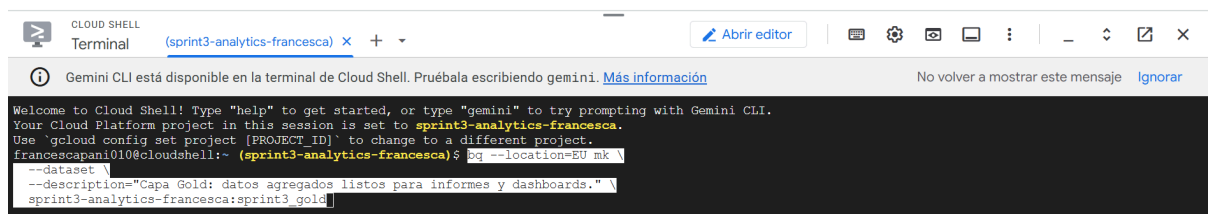




Notas sobre al código:

- **CREATE SCHEMA IF NOT EXISTS sprint3_silver** # creo el dataset con su nombre
- **OPTIONS (** # abro las opciones
- **location = 'EU',** # le digo donde guardarlo
- **description = 'Capa Silver: ...'** # añadido una pequeña descripción
- **);** # cierro y termino

Sigo con la creación de **sprint3_gold** con Cloud Shell:



```
CLOUD SHELL
Terminal (sprint3-analytics-francesca) x +
Abrir editor

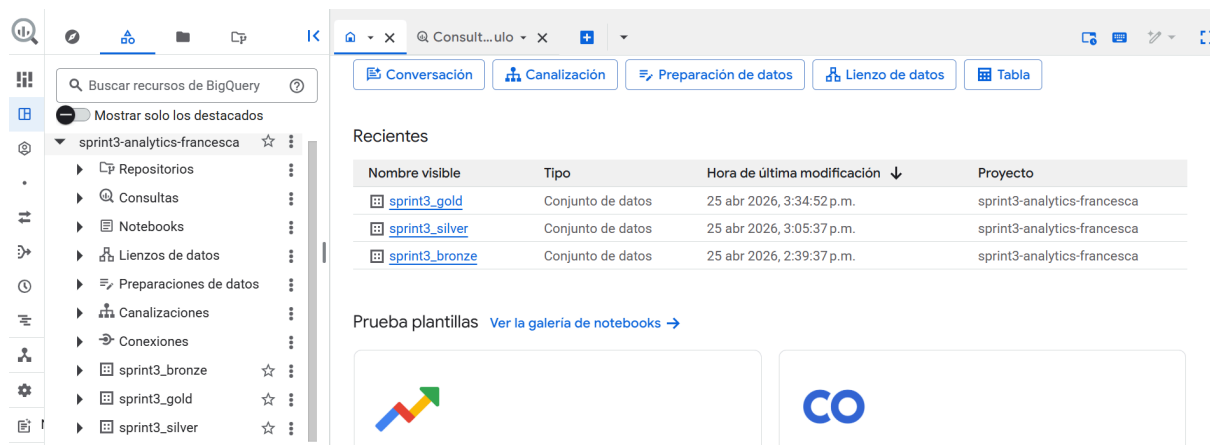
Gemini CLI está disponible en la terminal de Cloud Shell. Pruébala escribiendo gemini. Más información No volver a i

Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to sprint3-analytics-francesca.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
francescapani010@cloudshell:~ (sprint3-analytics-francesca)$ bq --location=EU mk \
--dataset \
--description="Capa Gold: datos agregados listos para informes y dashboards." \
sprint3-analytics-francesca:sprint3_gold
Dataset 'sprint3-analytics-francesca:sprint3_gold' successfully created.
francescapani010@cloudshell:~ (sprint3-analytics-francesca)$
```

Notas sobre el código:

- **bq --location=EU mk ** # le digo donde crear el dataset; mk = make para crear;
- **--dataset ** # Le dice a mk (make) qué crear: un dataset
- **--description="Capa Gold: datos..."** # descripción del dataset
- **sprint3-analytics-francesca:sprint3_gold** # destino y nombre del dataset

Los dataset se han creado correctamente:



Ejercicio 2: Ingesta en Capa Bronze (Conexión DDL)

Un analista profesional automatiza las labores. Ahora que tienes tu arquitectura a punto, poblaremos la primera capa (Bronze) sin mover los datos originales, creando Tablas Externas que apunten al Data Lake. Tarea: Escribe y ejecuta las sentencias CREATE EXTERNAL TABLE para conectar los siguientes archivos al dataset sprint3_bronze . Presta mucha atención a las "Notas Técnicas", ya que no todos los archivos tienen el mismo formato. Entrega: El código SQL utilizado para transactions_raw y una captura de pantalla con las 5 tablas creadas.

Mesa	Dominio	Ruta en la Nube (URI)	Notas Técnicas
transactions_raw	ERP	gs://bootcamp-data-analytics-public/ERP/transactions.csv	Delimitador: ; (Punto y coma)
companies_raw	ERP	gs://bootcamp-data-analytics-public/ERP/companies.csv	Cabecera: Ignora la 1a fila. Define manualmente el esquema porque todo es texto.
american_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/american_users.csv	Estándar.
european_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/european_users.csv	Estándar.
credit_cards_raw	CRM	gs://bootcamp-data-analytics-public/CRM/credit_cards.csv	Estándar.

Empiezo con la primera, **transactions_raw**.

- **CREATE EXTERNAL TABLE IF NOT EXISTS** `sprint3_bronze.transactions_raw` # crea una tabla externa, dentro del dataset `sprint3_bronze` con con "if not exists" para que el código sea reproducible
- **OPTIONS** (# abre el bloque de configuración
- **format** = 'CSV', # el archivo es un CSV
- **uris** = ['gs://bootcamp-data-analytics-public/ERP/transactions.csv'], # la ruta del archivo en Google Cloud Storage
- **field_delimiter** = ';', # separador punto y coma
- **skip_leading_rows** = 1 # le digo de saltar la primera fila*
-);

* Antes de definir la tabla externa, inspeccioné el archivo desde Cloud Shell con **"gcloud storage cat gs://bootcamp-data-analytics-public/ERP/transactions.csv | head -3"** para confirmar la estructura del CSV. Vi que la primera fila es la cabecera, lo que justifica el uso de **skip_leading_rows = 1** en las opciones de la tabla. Me parece una buena práctica que aplicaré antes de crear las demás tablas.

The screenshot shows a Google Cloud Shell window with a BigQuery query editor. The query is as follows:

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS sprint3_bronze.transactions_raw
2 OPTIONS (
3   format = 'CSV',
4   uris = ['gs://bootcamp-data-analytics-public/ERP/transactions.csv'],
5   field_delimiter = ';',
6   skip_leading_rows = 1
7 );
```

Below the query editor, a status message indicates: "Esta consulta procesará 0 B cuando se ejecute." (This query will process 0 B when executed).

The screenshot shows the same Google Cloud Shell window, but now the query has been executed. The left sidebar shows the project structure, with `sprint3_bronze` expanded and `transactions_raw` highlighted. The main panel shows the query execution results.

Se completó la consulta

Resultados de la consulta

Información del trabajo	
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_2abb8964_19dc51d5e0e (Ver detalles de rendimiento)
Usuario	francescapani010@gmail.com
Ubicación	EU
Hora de creación	25 abr 2026, 4:48:49 p.m. UTC+2
Hora de inicio	25 abr 2026, 4:48:49 p.m. UTC+2
Hora de finalización	25 abr 2026, 4:48:50 p.m. UTC+2
Duración	0 s
Bytes procesados	0 B
Bytes facturados	0 B

Sigo con **companies**:

```
Consulta sin título  Ejecutar Guardar Descargar Compartir
1 CREATE EXTERNAL TABLE IF NOT EXISTS sprint3_bronze.companies_raw (
2   company_id STRING,
3   company_name STRING,
4   phone STRING,
5   email STRING,
6   country STRING,
7   website STRING
8 )
9 OPTIONS (
10  format = 'CSV',
11  uris = ['gs://bootcamp-data-analytics-public/ERP/companies.csv'],
12  skip_leading_rows = 1
13 );
```

En este caso:

- el separador no lo inserto, ya que la coma es el delimitador por defecto de BigQuery;
- defino el esquema manualmente como pide el enunciado;
- Defino todas las columnas como STRING para preservar los datos crudos sin transformación, siguiendo la filosofía de la capa Bronze y según indica el enunciado ("todo es texto").

Se completó la consulta

Resultados de la consulta

Información del trabajo		Resultados	Detalles de la ejecución	Gráfico de ejecución
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_26358e78_19dc537194a	(Ver detalles de rendimiento)		
Usuario	francescapani010@gmail.com			
Ubicación	EU			
Hora de creación	25 abr 2026, 5:16:55 p.m. UTC+2			
Hora de inicio	25 abr 2026, 5:16:55 p.m. UTC+2			
Hora de finalización	25 abr 2026, 5:16:55 p.m. UTC+2			
Duración	0 s			
Bytes procesados	0 B			
Bytes facturados	0 B			

Para las 3 que quedan la query es igual, solo cambia el nombre de la tabla y la URI. Sigo con **American Users**:

Consulta sin título

Ejecutar Guardar Descargar

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS sprint3_bronze.american_users_raw
2 OPTIONS (
3   format = 'CSV',
4   uris = ['gs://bootcamp-data-analytics-public/CRM/american_users.csv'],
5   skip_leading_rows = 1
6 );
```

✓ Esta consulta procesará 0 B cuando se ejecute.

Buscar recursos de BigQuery

Mostrar solo los destacados

- sprint3-analytics-francesca
 - Repositorios
 - Consultas
 - Notebooks
 - Lienzos de datos
 - Preparaciones de datos
 - Canalizaciones
 - Conexiones
- sprint3_bronze
 - american_users_raw**
 - companies_raw
 - transactions_raw
- sprint3_gold
- sprint3_silver

Consulta sin título

Ejecutar Programa Abrir en Más Guardar

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS sprint3_bronze.american_users_raw
2 OPTIONS (
3   format = 'CSV',
4   uris = ['gs://bootcamp-data-analytics-public/CRM/american_users.csv'],
5   skip_leading_rows = 1
6 );
```

✓ Se completó la consulta

Resultados de la consulta

Información del trabajo	Resultados	Detalles de la ejecución	Gráfico de ejecución
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_53fa5bbb_19dc53e0f1c	(Ver detalles de rendimiento)	
Usuario	francescapani010@gmail.com		
Ubicación	EU		
Hora de creación	25 abr 2026, 5:24:31 p.m. UTC+2		
Hora de inicio	25 abr 2026, 5:24:31 p.m. UTC+2		
Hora de finalización	25 abr 2026, 5:24:32 p.m. UTC+2		
Duración	0 s		
Bytes procesados	0 B		
Bytes facturados	0 B		

Ahora con **European Users**:

Consulta sin título

Ejecutar Guardar Descargar

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS sprint3_bronze.european_users_raw
2 OPTIONS (
3   format = 'CSV',
4   uris = ['gs://bootcamp-data-analytics-public/CRM/european_users.csv'],
5   skip_leading_rows = 1
6 );
```

✓ Esta consulta procesará 0 B cuando se ejecute.

Buscar recursos de BigQuery

Mostrar solo los destacados

- sprint3-analytics-francesca
 - Repositorios
 - Consultas
 - Notebooks
 - Liensos de datos
 - Preparaciones de datos
 - Canalizaciones
 - Conexiones
- sprint3_bronze
 - american_users_raw
 - companies_raw
 - european_users_raw**
 - transactions_raw
- sprint3_gold
- sprint3_silver

Consulta sin título

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS sprint3_bronze.european_users_raw
2 OPTIONS (
3   format = 'CSV',
4   uris = ['gs://bootcamp-data-analytics-public/CRM/european_users.csv'],
5   skip_leading_rows = 1
6 );
```

Se completó la consulta

Resultados de la consulta

Información del trabajo	Resultados	Detalles de la ejecución	Gráfico de ejecución
ID de trabajo	sprint3-analytics-francesca EU bqjob_3d4bc9a4_19dc5405931	(Ver detalles de rendimiento)	
Usuario	francescapani01@gmail.com		
Ubicación	EU		
Hora de creación	25 abr 2026, 5:27:01 p.m. UTC+2		
Hora de inicio	25 abr 2026, 5:27:01 p.m. UTC+2		
Hora de finalización	25 abr 2026, 5:27:02 p.m. UTC+2		
Duración	0 s		
Bytes procesados	0 B		
Bytes facturados	0 B		

Y con Credit Cards:

Consulta sin título

Ejecutar Guardar Descargar

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS sprint3_bronze.credit_cards_raw
2 OPTIONS (
3   format = 'CSV',
4   uris = ['gs://bootcamp-data-analytics-public/CRM/credit_cards.csv'],
5   skip_leading_rows = 1
6 );
```

✓ Esta consulta procesará 0 B cuando se ejecute.

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS sprint3_bronze.credit_cards_raw
2 OPTIONS (
3   format = 'CSV',
4   uris = ['gs://bootcamp-data-analytics-public/CRM/credit_cards.csv'],
5   skip_leading_rows = 1
6 );
```

Se completó la consulta

Resultados de la consulta

Información del trabajo	
ID de trabajo	sprint3-analytics-francesca.EU.bqjob_2377c311_19dc543d5be (Ver detalles de rendimiento)
Usuario	francescapani010@gmail.com
Ubicación	EU
Hora de creación	25 abr 2026, 5:30:50 p.m. UTC+2
Hora de inicio	25 abr 2026, 5:30:50 p.m. UTC+2
Hora de finalización	25 abr 2026, 5:30:50 p.m. UTC+2
Duración	0 s
Bytes procesados	0 B
Bytes facturados	0 B
Milisegundos de ranura	0
Prioridad del trabajo	INTERACTIVE
Historial de trabajos	

Ejercicio 3: Carga de Datos Locales (Upload)

Falta el catálogo de Productos . Esta información no está en Data Lake, utiliza el archivo `products.csv` del sprint 2.

- Carga este archivo manualmente ("Upload") en el dataset `sprint3_bronze` y crea la tabla `products_raw`.
- Observa que ésta será la única Mesa Nativa de momento.

Esta es la configuración que he aplicado: formato CSV, delimitador de coma, una fila de cabecera omitida (abrí el documento para averiguar si hacía falta) y esquema generado por detección automática.

Resultado: tabla nativa `products_raw` creada en `sprint3_bronze`. Es la única tabla nativa del dataset; el resto son tablas externas que apuntan al Data Lake.

Crear tabla

Fuente

Create table from

Subir

Seleccionar archivo *

N3.Ex.2... products.csv

Formato del archivo

CSV

Destino

Proyecto *

sprint3-analytics-francesca

Conjunto de datos *

sprint3_bronze

Tabla *

products_raw

Crear tabla

Cancelar

Tipo de tabla

Tabla nativa

Esquema

☒ Detección automática

El esquema se generará automáticamente.

Configuración de partición

☒ Sin particionar

☐ Particionar por tiempo de transferencia

☐ Particionar por campo

Para particionar por campo, primero debes agregar campos al esquema anterior.

Opciones avanzadas

Preferencia de escritura

Agrega a la tabla

Cantidad de errores permitidos

0

☐ Valores desconocidos

Mapa de caracteres del nombre de la columna

Predeterminado

Delimitador de campos

Coma

Carácter de comilla

Comillas dobles

Coincidencia de columna de origen

Predeterminado

Filas del encabezado que se omitirán

1

☐ Saltos de línea en secciones entrecomilladas

☐ Filas irregulares

Crear tabla

Cancelar

sprint3_bronze

american_users_raw

companies_raw

credit_cards_raw

products_raw

transactions_raw

Mostrar más

Ejercicio 4: Arquitectura y Rendimiento. Materialización de Datos

a) Materialización de Datos (Asistido por IA)

Tarea: Crea una tabla nueva llamada `sprint3_bronze.transactions_raw_native` que sea una copia exacta de tu tabla externa `transactions_raw`.

Requisito de "AI-Pair Programming": No escribas el código SQL manualmente. Utiliza el Asistente de IA (Gemini) integrado en BigQuery (o el chat lateral) para generar la sentencia.

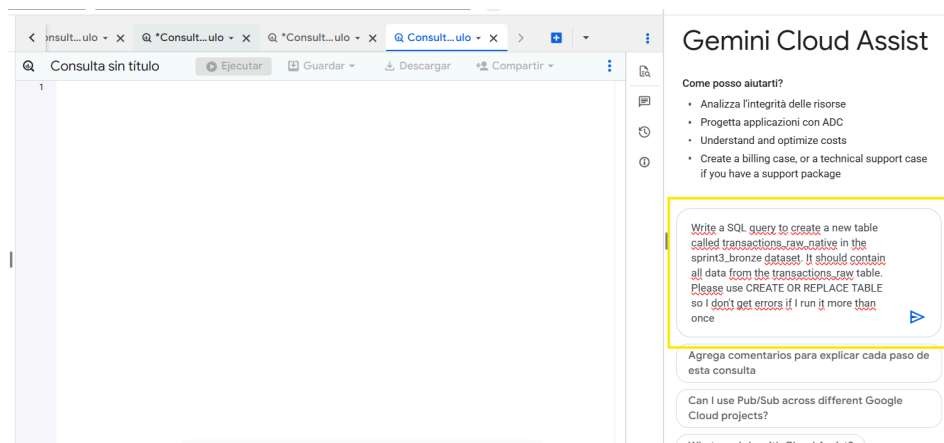
Tu Rol : Eres el responsable de validar que el código que te da la IA es correcto antes de ejecutarlo.

Prompt Sugerido: "Write a SQL query to create a new table called transactions_raw_native in the sprint3_bronze dataset. It should contain all data from the transactions_raw table. Please use CREATE OR REPLACE TABLE so I don't get errors if I run it more than once".

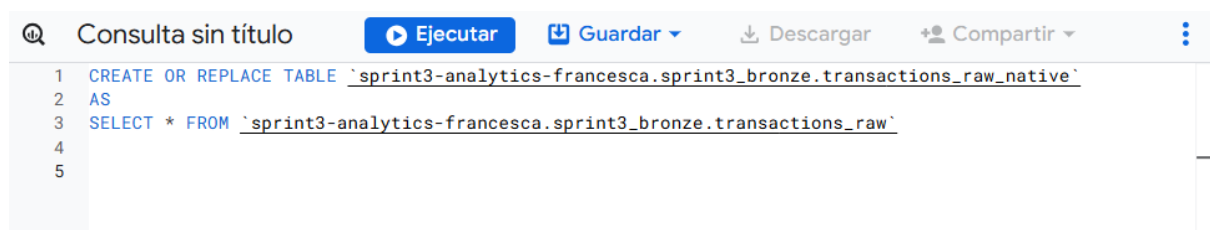
Entrega: Una captura de pantalla donde se vea:

- El prompt que escribiste en la IA.
- El código SQL que la IA va a generar.
- El resultado de la ejecución ("This statement created a new table...").

Inserto el prompt:



Me sugiere esta consulta:



- **CREATE OR REPLACE TABLE:** Crea la tabla. Si ya existía, la reemplaza. Esto es lo que pedí explícitamente en el prompt.
- Este el nombre completo de la tabla que sigue el orden proyecto.dataset.tabla :
`sprint3-analytics-francesca.sprint3\_bronze.transactions\_raw\_native`
- Crea la tabla con los resultado de este **SELECT: SELECT * FROM**
`sprint3-analytics-francesca.sprint3\_bronze.transactions\_raw`.

La consulta es correcta, le doy al botón Apply&Run:



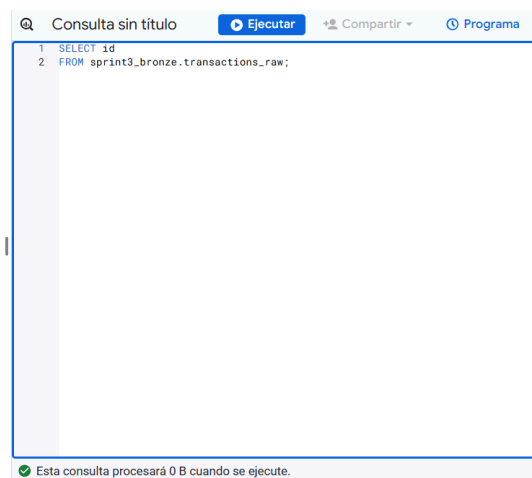
b) Auditoría de Costes

Tarea: Diseñar una prueba para comparar el coste de leer una sola columna (ej: id) en la tabla externa vs. la mesa nativa. Utiliza los Bytes procesados y los Bytes facturados para realizar la comparación.

- ¿Cuánto "pesa" la consulta en transactions_raw?
- ¿Cuánto "pesa" la misma consulta en transactions_raw_native?

Entrega: Capturas del validador mostrando la diferencia y una conclusión técnica.

Voy a hacer la misma query en dos sitios distintos: *sprint3_bronze.transactions_raw* y *sprint3_bronze.transactions_raw_native*. Mismo SELECT, misma columna, mismo proyecto y misma región. Solo cambia la tabla. Resultado antes de ejecutar la consulta:



Resultado después de ejecutar la consulta:

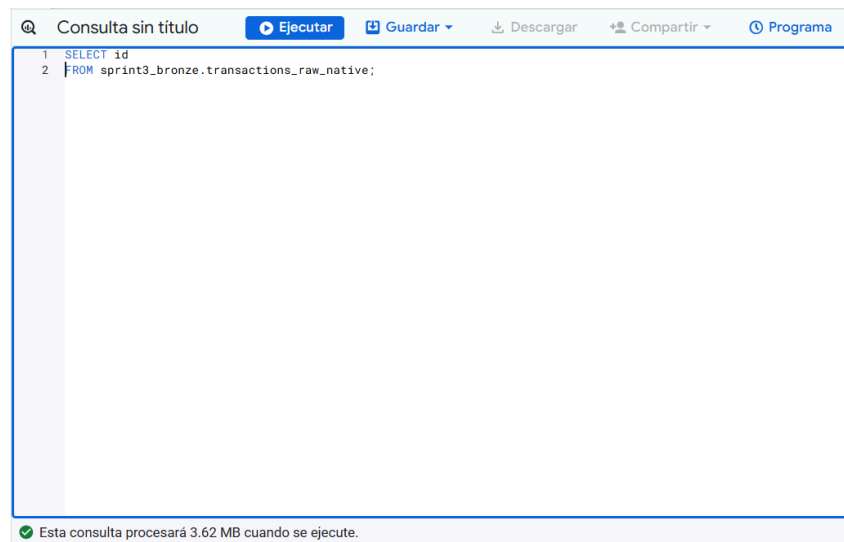
1	SELECT id
2	FROM sprint3_bronze.transactions_raw;

Se completó la consulta

Resultados de la consulta [Chat](#) [Guardar los resultados](#)

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_6ff0591b_19dc58b290e				Ver detalles de rendimiento
Usuario	francescapani010@gmail.com				
Ubicación	EU				
Hora de creación	25 abr 2026, 6:48:44 p.m. UTC+2				
Hora de inicio	25 abr 2026, 6:48:45 p.m. UTC+2				
Hora de finalización	25 abr 2026, 6:48:46 p.m. UTC+2				
Duración	1 s				
Bytes procesados	12.61 MB				
Bytes facturados	13 MB				
Millisegundos de ranura	1111				

Ahora paso a la otra tabla:



Resultado después de ejecutar la consulta:

1	SELECT id
2	FROM sprint3_bronze.transactions_raw_native;

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat Guardar los resultados

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_42ee907a_19dc58d8d68				
Usuario	francescapani010@gmail.com				
Ubicación	EU				
Hora de creación	25 abr 2026, 6:51:21 p.m. UTC+2				
Hora de inicio	25 abr 2026, 6:51:21 p.m. UTC+2				
Hora de finalización	25 abr 2026, 6:51:22 p.m. UTC+2				
Duración	1 s				
Bytes procesados	3.62 MB				
Bytes facturados	10 MB				

Conclusión

Al ejecutar la misma consulta (SELECT id) sobre dos tablas con los mismos datos pero distinto tipo de almacenamiento, se observa una diferencia significativa:

- Tabla externa (transactions_raw): 12.61 MB procesados / 13 MB facturados.
- Tabla nativa (transactions_raw_native): 3.62 MB procesados / 10 MB facturados.

* Bytes procesados son los que realmente lee BigQuery, los Bytes facturados son los que cobra.

En este ejemplo la tabla externa procesa casi 3.5 veces más bytes porque BigQuery debe leer el CSV completo desde Cloud Storage. La tabla nativa, en cambio, permite leer únicamente la columna solicitada (id), reduciendo drásticamente los bytes procesados. Es evidente como materializar las tablas en formato nativo reduce el coste de las consultas, especialmente cuando solo se accede a un subconjunto de columnas (que es además una best practice de optimización). En este sprint la diferencia parece pequeña (unos MB), pero en un entorno real con tablas de cientos de GB, esa

misma proporción se traduce en ahorros importantes. Además es evidente como el dry run del validador no estima correctamente el coste de las tablas externas (mostraba 0 B), por lo que el coste real sólo se confirmó tras la ejecución. En tablas nativas, el dry run sí coincide con el valor real procesado. Muy interesante como aunque la tabla nativa procese 3,5 veces menos, facture solo 30% menos (13 MB vs 10 MB), y esto pasa porque BigQuery factura un mínimo de 10 MB por cada query.

c) El peligro del LIMIT

Muchos analistas piensan que poner LIMIT 10 reduce el coste de la consulta. Tarea: Haz uno `SELECT * FROM transactions_raw_native LIMIT 10` sobre la tabla externa.

Comprobación : Mira el "Dry Run". ¿Ha bajado el coste respecto a no poner LIMIT? Explica brevemente por qué esto es una trampa para principiantes.

*Aquí tengo una duda: ¿el enunciado contiene una incoherencia entre el código (transactions_raw_native) y el texto explicativo (que indica “sobre la tabla externa”)? He optado por ejecutar la consulta sobre la tabla externa (transactions_raw), por ser coherente con el objetivo pedagógico del ejercicio: demostrar que el LIMIT no reduce el coste cuando los datos provienen de un CSV externo.

Escribo esta consulta, el validador dice “Esta consulta procesará 0 B cuando se ejecute”:

```
SELECT *  
  
FROM sprint3_bronze.transactions_raw  
  
LIMIT 10;
```

Realmente, al ejecutarla, este es el resultado:

Consulta sin título [Ejecutar] [Guardar] [Descargar] [Compartir] [Programa] [Abrir en] [Más]

```
1 SELECT *  
2 FROM sprint3_bronze.transactions_raw  
3 LIMIT 10;
```

✓ Esta consulta procesará 0 B cuando se ejecute.

Resultados de la consulta [Chat] [Guardar los resultados] [Abrir en]

Información del trabajo Resultados Visualización JSON Detalles de la ejecución Gráfico de ejecución

ID de trabajo	sprint3-analytics-francesca:EU.bqjob_1c423cc0_19dc5b32ec4 (Ver detalles de rendimiento)
Usuario	francescapani010@gmail.com
Ubicación	EU
Hora de creación	25 abr 2026, 7:32:27 p.m. UTC+2
Hora de inicio	25 abr 2026, 7:32:27 p.m. UTC+2
Hora de finalización	25 abr 2026, 7:32:27 p.m. UTC+2
Duración	0 s
Bytes procesados	1.44 KB
Bytes facturados	10 MB
Milisegundos de ranura	41

Al ejecutar `SELECT * FROM transactions_raw LIMIT 10` es evidente como los bytes procesados sean muy bajos (1.44 KB). Sin embargo, los bytes facturados son 10 MB, debido al mínimo de facturación que aplica BigQuery por consulta.

Creo que esta es la trampa, muchos creen que `LIMIT 10` reduce el coste, cuando en realidad:

- El ahorro real está limitado por el mínimo de 10 MB. Aunque BigQuery procese 1 KB, factura 10 MB igualmente.
- El `LIMIT` no es una estrategia de ahorro fiable.
- El dry run me muestra 0 B en tablas externas cuando hace la predicción, así que intuyo que no se pueda confiar en él al 100% para predecir el coste.

`LIMIT` es útil para ver muestras de datos rápidamente, pero no es una estrategia de optimización de costes. Para reducir costes, lo correcto sería seleccionar solo las columnas necesarias (por ejemplo: `SELECT col1, col2` en lugar de `SELECT *`) y filtrar con `WHERE`. Otra opción podría ser utilizar la vista previa. Además, según la documentación oficial de BigQuery, en tablas no agrupadas en clústeres (es decir tablas “desordenadas”) la cláusula `LIMIT` no reduce los costes de procesamiento: BigQuery factura por leer todas las columnas seleccionadas de la tabla completa, aunque solo se muestre un pequeño subconjunto de filas. La única excepción son las tablas clusterizadas, donde sí puede detener el escaneo al alcanzar el límite. Las tablas de este sprint no están clusterizadas, por lo que no aplica esta excepción.

Ejercicio 5: Adaptación de Sintaxis (Reporting)

Tu jefe quiere saber cuáles fueron los 5 días con mayores ingresos del año 2021 . Reto: Probablemente el campo `timestamp` es un `STRING`. Tendrás que investigar funciones de BigQuery (`SUBSTR`, `CAST`, `PARSE_TIMESTAMP`) para filtrar el año y agrupar por fecha correctamente.

```
1 SELECT DATE(timestamp) AS día, ROUND(SUM(amount),2) AS cantidad_ingresos_por_día
2 FROM sprint3_bronze.transactions_raw_native
3 WHERE EXTRACT(YEAR FROM timestamp) = 2021
4 AND declined = 0
5 GROUP BY DATE(timestamp)
6 ORDER BY cantidad_ingresos_por_día DESC
7 LIMIT 5;
```

✓ Esta consulta procesará 2.29 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_67301ce7_19dc5e48665 (Ver detalles de rendimiento)				
Usuario	francescapani010@gmail.com				
Ubicación	EU				
Hora de creación	25 abr 2026, 8:26:20 p.m. UTC+2				
Hora de inicio	25 abr 2026, 8:26:21 p.m. UTC+2				
Hora de finalización	25 abr 2026, 8:26:21 p.m. UTC+2				
Duración	0 s				
Bytes procesados	2.29 MB				
Bytes facturados	10 MB				

Algunas notas sobre el código:

- Uso **DATE(timestamp)** para extraer solo la fecha del campo TIMESTAMP, así puedo agrupar por día sin que la hora interfiera;
- Pongo **WHERE EXTRACT(YEAR FROM timestamp) = 2021** para filtrar el año;
- Pongo la condición **WHERE declined = 0** para excluir las transacciones declinadas ya que no serían ingresos reales;
- Agrupo por fecha con **GROUP BY**;
- Ordeno de mayor a menor, y pongo **LIMIT 5** para ver los 5 días con mayores ingresos del año 2021.

NOTA: El enunciado sugiere que el campo timestamp es un STRING. Esa pista del enunciado no aplica en mi caso, porque creo que cuando creé transactions_raw_native (con autodetect), BigQuery detectó la columna como TIMESTAMP automáticamente, no como STRING. Por eso me funciona **EXTRACT(YEAR FROM timestamp)** directamente.

Si la columna fuera STRING, habría tenido que hacer algo como:

WHERE EXTRACT(YEAR FROM PARSE_TIMESTAMP('%Y-%m-%d %H:%M:%S', timestamp)) = 2021

Aquí los resultados:

< Información del trabajo		Resultados	
Fila	dia ▼	cantidad_ingreso...	
1	2021-10-27	11440.81	
2	2021-01-22	11328.84	
3	2021-02-11	11182.16	
4	2021-04-29	11119.01	
5	2021-03-27	10898.51	

Ejercicio 6: Consultas Complejas

Necesitamos un informe que cruce datos. Tarea: Lista el nombre, país y fecha de las transacciones realizadas por empresas que realizaron operaciones entre 100 y 200 euros en alguna de estas fechas: 29-04-2015, 20-07-2018 o 13-03-2024.

Para resolver este ejercicio realizo un JOIN entre las tablas companies_raw y transactions_raw, conectando ambas a través del identificador de empresa (company_id en companies, business_id en transactions). Se aplican dos filtros combinados con AND en la cláusula WHERE:

- **BETWEEN 100 AND 200** para limitar el importe de la transacción.
- **IN ('2015-04-29', '2018-07-20', '2024-03-13')** para filtrar por las tres fechas concretas. Las fechas se escriben entre comillas para que BigQuery las interprete como STRING y luego las convierta a DATE.
- Finalmente, las ordeno por **valor_transaccion DESC**.

Consulta sin título

EjecutarProgramaAbrir enMásGuardarDescargarComp

```
1 SELECT c.company_name AS nombre_empresa, c.country AS pais, DATE(t.timestamp) AS fecha, ROUND(t.amount, 2) AS valor_transaccion
2 FROM sprint3_bronze.companies_raw c
3 JOIN sprint3_bronze.transactions_raw t
4 ON c.company_id = t.business_id
5 WHERE t.amount BETWEEN 100 AND 200
6 AND DATE(t.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
7 ORDER BY valor_transaccion DESC;
```

Se completó la consulta

Resultados de la consulta

Información del trabajoResultadosVisualizaciónJSONDetalles de la ejecuciónGráfico de ejecución

ID de trabajo	sprint3-analytics-francesca:EU.bquxjob_a50ec09_19dc63a9af8 (Ver detalles de rendimiento)
Usuario	francescapani010@gmail.com
Ubicación	EU
Hora de creación	25 abr 2026, 10:00:22 p.m. UTC+2
Hora de inicio	25 abr 2026, 10:00:23 p.m. UTC+2
Hora de finalización	25 abr 2026, 10:00:24 p.m. UTC+2
Duración	1 s
Bytes procesados	12.62 MB
Bytes facturados	20 MB
Milisegundos de ranura	8226

Y aquí van los resultados:

Resultados de la consulta				
Información del trabajoResultadosVisualizaciónJSONDetalles de la ejecución				
El almacenamiento en caché de metadatos está inhabilitado. Puedes acelerar las consultas en tablas ext				
Fila	nombre_empresa	pais	fecha	valor_transaccion
1	Donec Fringilla PC	France	2015-04-29	187.52
2	Netus Et Malesuada Ltd	Netherlands	2018-07-20	183.64
3	Auctor Mauris Vel LLP	United States	2015-04-29	181.42
4	Non Magna LLC	United Kingdom	2015-04-29	178.1
5	Dictum Eu Corp.	Canada	2015-04-29	177.67
6	Ac Libero Inc.	United Kingdom	2024-03-13	172.88
7	Amet Lorem LLP	Spain	2015-04-29	145.89
8	Magna Incorporated	New Zealand	2018-07-20	144.16
9	Justo Eu Arcu Ltd	Italy	2018-07-20	136.6
10	Aliquam Erat Volutpat LLP	Italy	2024-03-13	128.48
11	Aliquam Erat Volutpat LLP	Italy	2018-07-20	127.09
12	Ac Libero Inc.	United Kingdom	2015-04-29	124.04
13	Tempor Diam Institute	Netherlands	2018-07-20	124.04
14	Tempor Diam Institute	Netherlands	2018-07-20	124.04
15	At Pede Corp.	Italy	2018-07-20	115.16
16	Fusce Corp.	United States	2018-07-20	114.77
17	Et Magnis Ltd	Belgium	2018-07-20	114.44
18	Justo Eu Arcu Ltd	Italy	2024-03-13	114.09
19	Netus Et Malesuada Ltd	Netherlands	2018-07-20	103.73

Nivel 2: Limpieza y Transformación (ELT)

Ejercicio 1: Limpieza de Productos (Data Quality)

Crearemos la capa limpia ("Silver") para los productos. Tienes la tabla *sprint3_bronze.products_raw* cargada desde el CSV. El equipo de Data Governance te pide crear una mesa de productos limpia en *sprint3_silver.products_clean* que cumpla estas reglas de calidad:

- Estandarización de Nombres: La columna original id es ambigua; renombrarla a *product_id*. La columna *product_name* simplifícala a *name*.
- Limpieza de IDs : El campo *warehouse_id* tiene el formato antiguo "WH-4". Elimina el prefijo "WH-" y convierte el valor en número entero (INT64).
- Garantía de Precio : Asegúrate de que *price* es un número (FLOAT64), sin símbolos de moneda.
- Demás columnas : Conserva el campo *weight* (peso) tal cual.

Gracias a la arquitectura Medallion, no hace falta volver a subir el archivo CSV de products. De hecho la capa Silver lee los datos desde Bronze, los transforma y los guarda en una tabla nueva. Puedo hacer todo a través de una única query.

```
1 CREATE OR REPLACE TABLE sprint3_silver.products_clean AS
2 SELECT
3   id AS product_id,
4   product_name AS name,
5   SAFE_CAST(price AS FLOAT64) AS price,
6   colour,
7   weight,
8   SAFE_CAST(REPLACE(warehouse_id, 'WH-', '') AS INT64) AS warehouse_id
9 FROM sprint3_bronze.products_raw;
```

✓ Esta consulta procesará 5.62 KB cuando se ejecute.

Buscar recursos de BigQuery

Mostrar solo los destacados

- Canalizaciones
- Conexiones
- sprint3_bronze
 - american_users_raw
 - companies_raw
 - credit_cards_raw
 - european_users_raw
 - products_raw
 - transactions_raw
 - transactions_raw_native
- Mostrar más
- sprint3_gold
- sprint3_silver
 - products_clean

Consulta sin título

Ejecutar Programa Abrir en ▾

```
1 CREATE OR REPLACE TABLE sprint3_silver.products_clean AS
2 SELECT
3   id AS product_id,
4   product_name AS name,
5   SAFE_CAST(price AS FLOAT64) AS price,
6   colour,
7   weight,
8   SAFE_CAST(REPLACE(warehouse_id, 'WH-', '') AS INT64) AS warehouse_id
9 FROM sprint3_bronze.products_raw;
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico

ⓘ Esta declaración creó una nueva tabla con el nombre products_clean.

Algunas notas:

- Creo la tabla en Silver con **CREATE**. El **OR REPLACE** permite ejecutar la query varias veces sin error: si ya existe, la sobrescribe.
- Con el **SELECT** selecciono la lista de columnas que tendrá la nueva tabla.
- **id AS product_id**: renombro la columna;
- simplifico el nombre de la columna **product_name** que ahora es **name**;
- **SAFE_CAST(price AS FLOAT64) AS price**: la columna ya es FLOAT64 en Bronze (BigQuery lo detectó así), pero mantengo el SAFE_CAST para cumplir el enunciado y por si hay algún valor raro (si hay un dato raro, SAFE_CAST devuelve NULL en lugar de romper la query);
- color y weight se mantienen iguales;
- **SAFE_CAST(REPLACE(warehouse_id, 'WH-', '') AS INT64) AS warehouse_id**:
 - **REPLACE(warehouse_id, 'WH-', '')**: busca el prefijo 'WH-' y lo reemplaza por nada;
 - **SAFE_CAST(... AS INT64)**: convierte ese texto en número entero.

Ejercicio 2: Creación de Transacciones Limpias (Capa Silver)

Escenario: La tabla *transactions_raw* (Bronze) es insegura: el importe (amount) podría tener letras, las coordenadas podrían fallar y la fecha es un texto difícil de filtrar. Crearemos la mesa definitiva *sprint3_silver.transactions_clean* que garantice lo siguiente.

- Estandarización de Nombres: La columna original id es ambigua; renombrarla a *transaction_id*.
- La columna *product_name* simplifícala a *name* > **¿Error el enunciado? La tabla *transactions_raw* no tiene una columna que se llama *product_name*.**
- Robustez en Importes : Utiliza **SAFE_CAST** en el campo *amount*. Se falla, sustitúyelo por 0 (usando **IFNULL**).
- Fechas Reales : Convierte el campo *timestamp* (que es **STRING**) a tipo **TIMESTAMP** real.
- Coordenadas : Asegura que la *longitude* y *longitud* sean **FLOAT64** (utiliza **SAFE_CAST** para seguridad).
- Resto de campos : Manténlos igual.

Consulta sin título

Ejecutar
Guardar

```

1 CREATE OR REPLACE TABLE sprint3_silver.transactions_clean AS
2 SELECT
3   id AS transaction_id,
4   card_id,
5   business_id,
6   SAFE_CAST(timestamp AS TIMESTAMP) AS timestamp,
7   IFNULL(SAFE_CAST(amount AS FLOAT64), 0) AS amount,
8   declined,
9   ARRAY(
10     SELECT SAFE_CAST(TRIM(x) AS INT64)
11     FROM UNNEST(SPLIT(product_ids, ',')) AS x
12     WHERE TRIM(x) != ''
13   ) AS product_ids,
14   user_id,
15   SAFE_CAST(lat AS FLOAT64) AS lat,
16   SAFE_CAST(longitude AS FLOAT64) AS longitude
17 FROM sprint3_bronze.transactions_raw;

```

Esta consulta procesará 0 B cuando se ejecute.

Mostrar solo los destacados

- credit_cards_raw ☆ ⋮
- european_users_raw ☆ ⋮
- products_raw ☆ ⋮
- transactions_raw ☆ ⋮
- transactions_raw_native ☆ ⋮
- sprint3_gold ☆ ⋮
- v_marketing_kpis ☆ ⋮
- sprint3_silver ☆ ⋮
- companies_clean ☆ ⋮
- credit_cards_clean ☆ ⋮
- products_clean ☆ ⋮
- transactions_clean** ☆ ⋮
- users_combined ☆ ⋮

sprint3_silver.transactions_clean...

```

1 CREATE OR REPLACE TABLE sprint3_silver.transactions_clean AS
2 SELECT
3   id AS transaction_id,
4   card_id,
5   business_id,
6   SAFE_CAST(timestamp AS TIMESTAMP) AS timestamp,
7   IFNULL(SAFE_CAST(amount AS FLOAT64), 0) AS amount,
8   declined,
9   ARRAY(
10     SELECT SAFE_CAST(TRIM(x) AS INT64)
11     FROM UNNEST(SPLIT(product_ids, ',')) AS x
12     WHERE TRIM(x) != ''
13   ) AS product_ids,
14   user_id,
15   SAFE_CAST(lat AS FLOAT64) AS lat,
16   SAFE_CAST(longitude AS FLOAT64) AS longitude
17 FROM sprint3_bronze.transactions_raw;
```

✓ Se completó la consulta

Resultados de la consulta

Información del trabajo		Resultados	Detalles de la ejecución
ID de trabajo	sprint3-analytics-francesca:EU.bquxjob_7c9b11fb		
Usuario	francescapani010@gmail.com		
Ubicación	EU		
Hora de creación	27 abr 2026, 8:35:34 p.m. UTC+2		
Hora de inicio	27 abr 2026, 8:35:34 p.m. UTC+2		
Hora de finalización	27 abr 2026, 8:35:37 p.m. UTC+2		
Duración	2 s		
Bytes procesados	12.61 MB		
Bytes facturados	13 MB		

 sprint3_silver.transactions_clea... Ejecutar

```
1 CREATE OR REPLACE TABLE sprint3_silver.transactions_clean AS
2 SELECT
3     id AS transaction_id,
4     card_id,
5     business_id,
6     SAFE_CAST(timestamp AS TIMESTAMP) AS timestamp,
7     IFNULL(SAFE_CAST(amount AS FLOAT64), 0) AS amount,
8     declined,
9     ARRAY(
10         SELECT SAFE_CAST(TRIM(x) AS INT64)
11         FROM UNNEST(SPLIT(product_ids, ',')) AS x
12         WHERE TRIM(x) != ''
13     ) AS product_ids,
14     user_id,
15     SAFE_CAST(lat AS FLOAT64) AS lat,
16     SAFE_CAST(longitude AS FLOAT64) AS longitude
17 FROM sprint3_bronze.transactions_raw;
```

Se completó la consulta

Resultados de la consulta

Información del trabajo	Resultados	Detalles de la ejecución
ID de trabajo	sprint3-analytics-francesca:EU.bqxxjob_7c9b11fb	
Usuario	francescapani010@gmail.com	
Ubicación	EU	
Hora de creación	27 abr 2026, 8:35:34 p.m. UTC+2	
Hora de inicio	27 abr 2026, 8:35:34 p.m. UTC+2	
Hora de finalización	27 abr 2026, 8:35:37 p.m. UTC+2	
Duración	2 s	
Bytes procesados	12.61 MB	
Bytes facturados	13 MB	

Notas:

- La columna timestamp NO era STRING, ya era TIMESTAMP. BigQuery cuando he creado la tabla la detectó como timestamp directamente. Mantengo el **SAFE_CAST** como garantía. Si no lo hubiese detectado, habría quedado así:
 - SAFE.PARSE_TIMESTAMP('%Y-%m-%d %H:%M:%S', timestamp) AS timestamp.**

En la misma consulta, aunque el enunciado no lo pida claramente, ya que estamos en la capa Silver que es donde se hace “limpieza”, convierto `product_ids` a array. En el detalle:

- **ARRAY:** empieza a construir un ARRAY nuevo
- **SELECT SAFE_CAST(TRIM(x) AS INT64):**
 - **x** = cada valor del array tras el split
 - **TRIM(x):** elimina espacios
 - **SAFE_CAST(... AS INT64):** convierte a número y si falla devuelve NULL en vez de error
- **FROM UNNEST(SPLIT(product_ids, ',')) AS x**
 - **SPLIT(product_ids, ','):** convierte el string en array: es decir desde "17, 66, 3" a ['17', ' 66', ' 3']
 - **UNNEST(...):** Convierte ese array en filas
 - **AS x:** Le da nombre a cada elemento
- **WHERE TRIM(x) != "":** filtra los elementos vacíos.

- **UNNEST(...):** Convierte ese array en filas

- **AS x:** Le da nombre a cada elemento

Ejercicio 3: Unificación de Usuarios (UNION)

Tenemos a los usuarios fragmentados por región. Tarea: Crea la tabla `sprint3_silver.users_combined`. Utiliza **UNION ALL** para unificar a los usuarios de EE.UU. y Europa en una única lista maestra. Añade una columna calculada `origin` para saber de dónde vienen.

Estandarización de Nombres: La columna original `id` es ambigua; renombrarla a `user_id`.

```
1 CREATE OR REPLACE TABLE sprint3_silver.users_combined AS
2 SELECT
3   id AS user_id,
4   name,
5   surname,
6   phone,
7   email,
8   birth_date,
9   country,
10  city,
11  postal_code,
12  address,
13  'Europe' AS origin
14 FROM sprint3_bronze.european_users_raw
15
16 UNION ALL
17
18 SELECT
19   id AS user_id,
20   name,
21   surname,
22   phone,
23   email,
24   birth_date,
25   country,
26   city,
27   postal_code,
28   address,
29   'America' AS origin
30 FROM sprint3_bronze.american_users_raw;
```

Esta consulta procesará 0 B cuando se ejecute.

Buscar recursos de BigQuery

Mostrar solo los destacados

- Preparaciones de datos
- Canalizaciones
- Conexiones
- sprint3_bronze
 - american_users_raw
 - companies_raw
 - credit_cards_raw
 - european_users_raw
 - products_raw
 - transactions_raw
 - transactions_raw_native
- Mostrar más
- sprint3_gold
- sprint3_silver
 - products_clean
 - transactions_clean
 - users_combined**

```
1 CREATE OR REPLACE TABLE sprint3_silver.users_combined AS
2 SELECT
3   id AS user_id,
4   name,
5   surname,
6   phone,
7   email,
8   birth_date,
9   country,
10  city,
11  postal_code,
12  address,
13  'Europe' AS origin
14 FROM sprint3_bronze.european_users_raw
15
16 UNION ALL
17
18 SELECT
19   id AS user_id,
20   name,
21   surname,
22   phone,
23   email,
24   birth_date,
25   country,
26   city,
27   postal_code,
28   address,
29   'America' AS origin
30 FROM sprint3_bronze.american_users_raw;
```

Se completó la consulta

Resultados de la consulta

Información del trabajo	Resultados	Detalles de la ejecución	Gráfico
Hora de inicio	27 abr 2026, 4:14:17 p.m. UTC+2		
Hora de finalización	27 abr 2026, 4:14:19 p.m. UTC+2		
Duración	1 s		
Bytes procesados	568.04 KB		
Bytes facturados	20 MB		

Notas:

- Utilizo **UNION ALL** (y no **UNION DISTINCT**) porque los usuarios de las dos regiones son personas distintas. Lo que hace es pegar los dos **SELECT** en una única tabla ;

- En ambos caso, hago un renombrado de id genérico a **user_id** específico;
- Añado una columna llamada “**origin**” para guardar el origen de los usuarios;

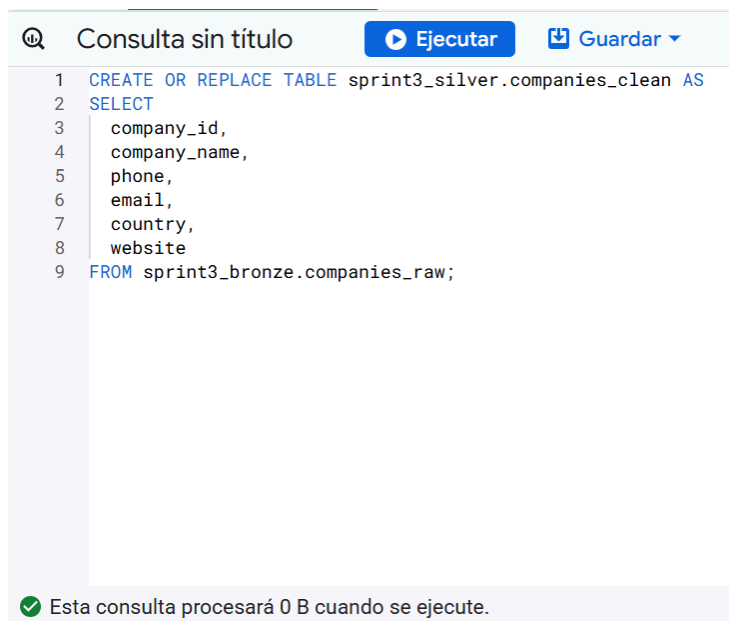
Ejercicio 4: Materialización de Compañías y Tarjetas de Crédito

Para completar el modelo de datos en la capa Silver, nos faltan dos dimensiones clave que actualmente dependen de archivos CSV externos: Compañías y Tarjetas de Crédito. No podemos depender de archivos sueltos para el análisis final. Debemos importar estos datos en tablas nativas (Silver) y aprovechar para corregir formatos.

Tareas:

- Crea la tabla *sprint3_silver.companies_clean*. Copia los datos tal cual, pero asegúrate de que sea una tabla nativa de BigQuery (no una vista externa).
- Crea la tabla *sprint3_silver.credit_cards_clean*. Copia los datos tal cual, pero asegúrate de que sea una tabla nativa de BigQuery (no una vista externa).
- Si es necesario, renombrar el id según corresponda.

Para garantizar la independencia respecto a los archivos CSV de origen como pide el enunciado, creo en Silver las dos tablas. La operación es directa (lo importante es la palabra TABLE, sin "EXTERNAL" como en los primeros ejercicios): se leen los datos desde las tablas externas de Bronze y se guardan físicamente en tablas nativas de Silver. Empiezo con *sprint3_silver.companies_clean*:



The screenshot shows a BigQuery console interface. At the top, there's a search bar with a magnifying glass icon and the text "Consulta sin título". To the right of the search bar are two buttons: "Ejecutar" (Execute) with a play icon and "Guardar" (Save) with a download icon. Below the search bar is a text area containing a SQL query. The query is as follows:

```
1 CREATE OR REPLACE TABLE sprint3_silver.companies_clean AS
2 SELECT
3   company_id,
4   company_name,
5   phone,
6   email,
7   country,
8   website
9 FROM sprint3_bronze.companies_raw;
```

At the bottom of the console, there is a status bar with a green checkmark icon and the text: "Esta consulta procesará 0 B cuando se ejecute."

Buscar recursos de BigQuery

Mostrar solo los destacados

- Preparaciones de datos
- Canalizaciones
- Conexiones
- sprint3_bronze
 - american_users_raw
 - companies_raw
 - credit_cards_raw
 - european_users_raw
 - products_raw
 - transactions_raw
 - transactions_raw_native
- Mostrar más
- sprint3_gold
- sprint3_silver
 - companies_clean**
 - products_clean
 - transactions_clean
 - users_combined

Consulta sin título

```
1 CREATE OR REPLACE TABLE sprint3_silver.companies_clean AS
2 SELECT
3   company_id,
4   company_name,
5   phone,
6   email,
7   country,
8   website
9 FROM sprint3_bronze.companies_raw;
```

Se completó la consulta

Resultados de la consulta

Información del trabajo	Resultados	Detalles de la ejecución	Gráfico de e
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_761ad49_19dcf6d4db0	Ver	
Usuario	francescapani010@gmail.com		
Ubicación	EU		
Hora de creación	27 abr 2026, 4:52:19 p.m. UTC+2		
Hora de inicio	27 abr 2026, 4:52:20 p.m. UTC+2		
Hora de finalización	27 abr 2026, 4:52:21 p.m. UTC+2		
Duración	1 s		
Bytes procesados	10.1 KB		
Bytes facturados	10 MB		

Sigo con **sprint3_silver.credit_cards_clean**:

Consulta sin título

```
1 CREATE OR REPLACE TABLE sprint3_silver.credit_cards_clean AS
2 SELECT
3   id AS credit_card_id,
4   user_id,
5   iban,
6   pan,
7   pin,
8   cvv,
9   track1,
10  track2,
11  expiring_date
12 FROM sprint3_bronze.credit_cards_raw;
```

Esta consulta procesará 0 B cuando se ejecute.

The screenshot shows the Google BigQuery interface. On the left, a sidebar lists various datasets, with `sprint3_silver.credit_cards_clean` highlighted. The main area displays a SQL query titled 'Consulta sin título' with the following code:

```
1 CREATE OR REPLACE TABLE sprint3_silver.credit_cards_clean AS
2 SELECT
3   id AS credit_card_id,
4   user_id,
5   iban,
6   pan,
7   pin,
8   cvv,
9   track1,
10  track2,
11  expiring_date
12 FROM sprint3_bronze.credit_cards_raw;
```

Below the query, a status message indicates 'Se completó la consulta'. The 'Resultados de la consulta' section shows a table with job information:

Información del trabajo	
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_7190530f_
Usuario	francescapani010@gmail.com
Ubicación	EU
Hora de creación	27 abr 2026, 4:59:22 p.m. UTC+2
Hora de inicio	27 abr 2026, 4:59:22 p.m. UTC+2
Hora de finalización	27 abr 2026, 4:59:24 p.m. UTC+2
Duración	1 s
Bytes procesados	804.85 KB
Bytes facturados	10 MB

Nivel 3: Presentación de Datos y Creación de Vistas

Ejercicio 1: La Vista de Marketing (Lógica de Negocio)

Marketing necesita segmentar a los clientes corporativos, pero no saben hacer JOINS. Su tarea es dejarles la información preparada. Tarea: Crea una vista llamada `sprint3_gold.v_marketing_kpis` que muestre la siguiente información para cada compañía:

- Nombre de la compañía, Teléfono y País (origen: `companies_clean`).
- Media de compra (AVG(amount) de `transactions_clean`).
- Clasificación de Cliente (Lógica):
 - Crea una columna calculada llamada `client_tier`.
 - Si la media de compra es superior a 260 €, etiqueta como "Premium".
 - Si es inferior, etiqueta como "Standard".

Entrega: Realiza una consulta `SELECT` *sobre tu nueva vista, ordenando los resultados para que aparezcan primero los clientes "Premium" y, dentro de éstos, los que tengan mayor promedio de compra.

Primero, creo la vista llamada `sprint3_gold.v_marketing_kpis`:

- la creo con `CREATE OR REPLACE VIEW...AS`;
- pongo `WHERE t.declined = 0` con el objetivo de excluir las transacciones declinadas). Creo que tiene sentido para calcular la "media de compra" real;
- el `CASE`: clasifica cada empresa como "Premium" si su media supera 260 € y como "Standard" en caso contrario;
- Esta primera query es la que crea la vista.

```
1 CREATE OR REPLACE VIEW sprint3_gold.v_marketing_kpis AS
2 SELECT
3   c.company_name AS nombre_empresa,
4   c.phone AS telefono,
5   c.country AS pais,
6   ROUND(AVG(t.amount),2) AS media_de_compra,
7 CASE
8   WHEN ROUND(AVG(t.amount),2) > 260 THEN 'Premium'
9   ELSE 'Standard'
10 END AS client_tier
11 FROM sprint3_silver.companies_clean c
12 JOIN sprint3_silver.transactions_clean t
13 ON c.company_id = t.business_id
14 WHERE t.declined = 0
15 GROUP BY c.company_id, c.company_name, c.phone, c.country;
```

✓ Esta consulta procesará 0 B cuando se ejecute.

Buscar recursos de BigQuery

Mostrar solo los destacados

companies_raw

credit_cards_raw

european_users_raw

products_raw

transactions_raw

transactions_raw_native

sprint3_gold

v_marketing_kpis

sprint3_silver

companies_clean

credit_cards_clean

products_clean

transactions_clean

users_combined

Consulta sin título

Ejecutar

Compartir

Pr

```
1 CREATE OR REPLACE VIEW sprint3_gold.v_marketing_kpis AS
2 SELECT
3   c.company_name AS nombre_empresa,
4   c.phone AS telefono,
5   c.country AS pais,
6   ROUND(AVG(t.amount),2) AS media_de_compra,
7 CASE
8   WHEN ROUND(AVG(t.amount),2) > 260 THEN 'Premium'
9   ELSE 'Standard'
10 END AS client_tier
11 FROM sprint3_silver.companies_clean c
12 JOIN sprint3_silver.transactions_clean t
13 ON c.company_id = t.business_id
14 WHERE t.declined = 0
15 GROUP BY c.company_id, c.company_name, c.phone, c.country;
```

Se completó la consulta

Resultados de la consulta

Información del trabajo	Resultados	Detalles de la ejecución
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_23df131b_19dc	
Usuario	francescapani010@gmail.com	
Ubicación	EU	
Hora de creación	27 abr 2026, 5:52:01 p.m. UTC+2	
Hora de inicio	27 abr 2026, 5:52:02 p.m. UTC+2	
Hora de finalización	27 abr 2026, 5:52:02 p.m. UTC+2	
Duración	0 s	
Bytes procesados	0 B	
Bytes facturados	0 B	

Ahora que está creada, realiza una consulta `SELECT *` sobre la vista, ordenando los resultados - primero los clientes "Premium" y, dentro de éstos, los con mayor promedio de compra.

```
1 SELECT *
2 FROM sprint3_gold.v_marketing_kpis
3 ORDER BY
4   client_tier,
5   media_de_compra DESC;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON	Detalles
ID de trabajo	sprint3-analytics-francesca:EU.bqjob_e955ad7_19dcfaec170			
Usuario	francescapani010@gmail.com			
Ubicación	EU			
Hora de creación	27 abr 2026, 6:03:49 p.m. UTC+2			
Hora de inicio	27 abr 2026, 6:03:49 p.m. UTC+2			
Hora de finalización	27 abr 2026, 6:03:49 p.m. UTC+2			
Duración	0 s			
Bytes procesados	2.29 MB			
Bytes facturados	20 MB			

Y aquí los primeros 16 resultados:

1	SELECT *
2	FROM sprint3_gold.v_marketing_kpis
3	ORDER BY
4	client_tier,
5	media_de_compra DESC;

✓ Esta consulta procesará 2.29 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	nombre_empresa	telefono	pais	media_de_compra	client_tier
1	Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.91	Premium
2	Pretium Neque Corp.	07 77 48 55 28	Australia	275.58	Premium
3	Urna Convallis Associates	06 01 24 77 04	United States	273.57	Premium
4	At Associates	09 56 61 10 65	New Zealand	272.74	Premium
5	Metus Vitae Associates	08 25 44 40 66	Australia	270.05	Premium
6	Aliquet Diam Limited	02 76 61 47 46	United States	269.29	Premium
7	Nec Luctus LLC	02 14 71 75 73	Norway	268.6	Premium
8	Neque Tellus Incorporated	04 43 18 34 19	Ireland	267.56	Premium
9	Cras Consulting	07 50 10 85 63	Belgium	267.38	Premium
10	Sed LLC	01 63 16 26 52	Belgium	266.61	Premium
11	Eget Ipsum Ltd	03 67 44 56 72	United States	266.27	Premium
12	Tortor Nunc Commodo Company	05 35 92 77 16	United States	266.26	Premium
13	Amet Institute	06 33 40 21 33	Australia	265.54	Premium
14	Fusce Corp.	08 14 97 58 85	United States	265.12	Premium
15	Ut Semper Foundation	01 60 36 33 06	Sweden	264.73	Premium
16	Orci Adipiscing Limited	03 18 00 77 81	United Kingdom	264.59	Premium

Ejercicio 2: Ranking de Productos (La Potencia de los Arrays)

Escenario de Negocio: El equipo de ventas desea optimizar el catálogo. Necesitan un informe en la capa Gold que muestre el rendimiento real de cada producto. Gracias a tu buena ingeniería en la capa Silver, la mesa `transactions_clean` ya tiene los IDs de producto perfectamente organizados en una lista (Array). Ahora toca explotar esa estructura. **Objetivo:** Crea la tabla `sprint3_gold.product_sales_ranking` que contenga el inventario completo de productos y cuántas veces se ha vendido cada uno.

Requisitos del Informe:

- **Detalle del Producto :** Debe incluir `product_id`, `name`, `price` y `color` (vienen de la tabla `products_clean`).
- **Métrica de Negocio :** Una columna nueva `total_sold` que cuente cuántas veces aparece este producto en las transacciones.
- **Integridad :** Deben aparecer todos los productos, incluso los que tienen 0 ventas (quizás es necesario descatalogarlos).

Pista Técnica:

- En `transactions_clean`, los productos están "encapsulados" dentro de un Array por cada transacción.
- Primero necesitarás "allanar" la tabla de transacciones usando `UNNEST(product_ids)` para poder contar los productos individualmente.
- Después, haz un cruce (`LEFT JOIN`) empezando por la mesa de productos para no perder aquellos que nunca se han vendido.

Buscar recursos de BigQuery

Mostrar solo los destacados

- Los 5 días con mayores ir
- SELECT * FROM `sprint3`
- Sprint3_gold.product_sale
- Sprint3_gold.product_sale**
- Sprint3_gold.v_marketing
- Sprint3_silver.companies
- Sprint3_silver.credit_card
- Sprint3_silver.products_cl
- Sprint3_silver.transaction
- Sprint3_silver.users_comt
- Tabla nueva sprint3_bron
- sprint3_gold.product_sale
- sprint3_silver.transaction

Notebooks

Lienzos de datos

Preparaciones de datos

Canalizaciones

Conexiones

sprint3_bronze

sprint3_gold

product_sales_ranking

v_marketing_kpis

🔍 Sprint3_gold.product_sales_ra...
Ejecutar

```

1 CREATE OR REPLACE TABLE sprint3_gold.product_sales_ranking AS
2 WITH ventas_por_producto AS (
3   SELECT
4     p AS product_id,
5     COUNT(*) AS total_sold
6   FROM sprint3_silver.transactions_clean t
7   CROSS JOIN UNNEST(t.product_ids) AS p
8   GROUP BY p
9 )
10 SELECT
11   p.product_id,
12   p.name,
13   p.price,
14   p.colour,
15   IFNULL(v.total_sold, 0) AS total_sold
16 FROM sprint3_silver.products_clean p
17 LEFT JOIN ventas_por_producto v
18   ON p.product_id = v.product_id;

```

✅ Esta consulta procesará 1.94 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Detalles de la ejecución
ID de trabajo	sprint3-analytics-francesca:EU.bquxjob_7a6616dc_1	
Usuario	francescapani010@gmail.com	
Ubicación	EU	
Hora de creación	28 abr 2026, 4:29:40 p.m. UTC+2	
Hora de inicio	28 abr 2026, 4:29:40 p.m. UTC+2	
Hora de finalización	28 abr 2026, 4:29:43 p.m. UTC+2	
Duración	2 s	
Bytes procesados	1.94 MB	
Bytes facturados	20 MB	

Notas sobre este código:

- **CREATE OR REPLACE TABLE sprint3_gold.product_sales_ranking AS:** Crea la tabla en Gold.
- **WITH ventas_por_producto AS:** Abre la CTE (tabla temporal) con nombre ventas_por_producto. Lo que vaya entre paréntesis hasta la línea 8 será una tabla temporal;
- **SELECT** dos columnas:
 - **p AS product_id:** renombra la variable del unnest de abajo

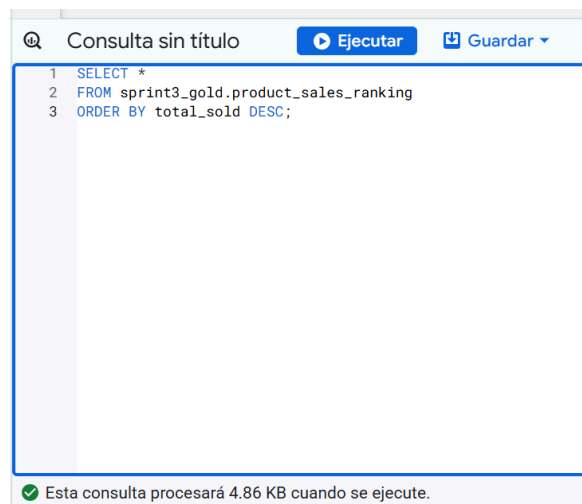
- **COUNT(*) AS total_sold**: Cuenta cuántas filas hay en cada grupo (los grupos los define el GROUP BY de abajo)
- **FROM sprint3_silver.transactions_clean t**: lee de la tabla de transacciones, con alias t
- **CROSS JOIN UNNEST(t.product_ids) AS p**: Explota el array product_ids en filas individuales. Cada elemento se llama p.
- **GROUP BY p**: Agrupa por producto, para que COUNT(*) cuente por separado cada uno.
- **SELECT principal**: Las 4 columnas del producto y el conteo de ventas.
- **IFNULL(v.total_sold, 0) AS total_sold**: Coge total_sold de la CTE. Si es NULL (porque el producto no tuvo ventas y el LEFT JOIN no encontró match), lo reemplaza por 0;
- **FROM sprint3_silver.products_clean p**: Lee la tabla de productos (alias p).
- **LEFT JOIN ventas_por_producto v**: mantiene todas las filas de la tabla izquierda (products_clean), incluso si no encuentran match en la derecha (ventas_por_producto). Con otro JOIN, los productos sin ventas habrían desaparecido.

En resumen:

- CTE: cuenta cuántas veces aparece cada producto en transacciones (después de aplanar el array con UNNEST). NB: la CTE solo contiene los productos que se vendieron al menos una vez.
- CROSS JOIN UNNEST(t.product_ids): cada fila se combina solo con su propio array, no con todos los arrays (que es lo que haría normalmente un cross join).
- Query principal: coge todos los productos.
- LEFT JOIN: incluye en el conteo los que los que tienen 0 ventas. O mejor dicho: mantiene todos los productos aunque no tengan match en la CTE.
- IFNULL: los que no tienen ventas se quedan con 0.

Ejercicio 3: Exportación de Resultados

Tu manager no tiene acceso a BigQuery y quiere el listado de "Top Productos" en un Excel para una reunión. Tarea: Exporta los datos de la tabla product_sales_ranking a Google Sheets o descarga el archivo CSV localmente. Entrega: Una captura de pantalla donde se vea el archivo abierto (Excel/Sheets) con los datos correctamente formateados.



Consulta sin título

```

1 SELECT *
2 FROM sprint3_gold.product_sales_ranking
3 ORDER BY total_sold DESC;

```

✓ Esta consulta procesará 4.86 KB cuando se ejecute.

Top Productos File Modifica Visualizza Inserisci Formato Dati Strumenti Estensioni Guida Menu 100% € % .0 .00 123 Prede... 10						
I4						
	A	B	C	D	E	F
1	product_id	name	price	colour	total_sold	
2	52	riverlands the duel	31.52.00	#727272	2654	
3	29	Tully maester Tarly	167.2	#111111	2635	
4	21	duel Direwolf	96.9	#e2e2e2	2609	
5	16	the duel warden	181.31.00	#666666	2608	
6	66	mustafar jinn	2.12	#545454	2601	
7	87	sith Jade	96.26.00	#e2e2e2	2598	
8	48	rock Renly in	21.53	#9e9e9e	2597	
9	33	duel warden	127.09.00	#191919	2597	
10	23	riverlands north	170.36.00	#545454	2593	
11	68	Stark Karstark	167.37.00	#333333	2589	
12	88	Stannis warden south	167.15.00	#515151	2587	
13	4	warden south duel	72.29.00	#111111	2584	
14	73	Dorne bastard	114.09.00	#848484	2584	
15	28	chewbacca mustafar	127.44.00	#efefef	2584	
16	34	of north	128.48.00	#919191	2580	
17	31	Lannister	85.02.00	#3f3f3f	2580	
18	38	ewok sith	55.43.00	#666666	2578	
19	61	Winterfell Lannister	28.01.00	#848484	2575	
20	54	dooku solo	13.14	#e5e5e5	2575	
21	36	Lannister	96.22.00	#a5a5a5	2575	
22	24	south duel tourney	49.39.00	#aaaaaa	2574	
23	2	Tarly Stark	9.24	#919191	2573	
24	11	Karstark Dorne	49.7	#141414	2573	
25	72	north	161.6	#474747	2573	
26	39	warden	197.05.00	#a0a0a0	2571	
27	10	Karstark Dorne	119.52.00	#f4f4f4	2571	
28	47	Tully	82.15.00	#919191	2571	
29	74	duel warden	158.29.00	#707070	2569	
30	64	Direwolf	196.02.00	#b7b7b7	2568	
31	58	solo palpatine chewbacca	130.59.00	#2b2b2b	2567	
32	65	riverlands the	122.31.00	#6b6b6b	2567	
33	53	kingsblood Littlefinger the	138.21.00	#3a3a3a	2564	

En la consulta he ordenado por total_sold DESC para presentar los productos más vendidos en primer lugar (formato "Top Productos"). Luego he exportado el resultado a Google Sheets, generando [un archivo accesible](#).

Corregido con Ángel Herrezuelo Hernández